



Rule-Based Semantic Validation for Standardized Linked Building Models

Philipp Hagedorn^(✉)  and Markus König 

Computing in Engineering, Ruhr-University Bochum, Universitätsstr. 150,
44801 Bochum, Germany
philipp.hagedorn-n6v@rub.de

Abstract. Planning data of construction projects consist of distributed data in the manner of building models and accompanying heterogeneous information models. For data drops, these models can be stored and linked within an information container using link sets. When using open standards and file formats, link sets are often created and maintained manually, because an automated traceable generation of links is not yet reasonably provided. In this research, semi-automatic validation is preferred to ensure the syntactic, semantic and technical correctness of link sets for the practical application of so-called linked building models. Therefore, rule validation approaches in the AEC industry are examined regarding their feasibility for linked building models. A method for rule checking is presented and evaluated in a case study, which utilizes a building model of a fictive tunnel construction project and a linked tunnel construction schedule as a linked building model inside an Information Container for Data Drop.

Keywords: BIM · Linked building models · Information Container for Data Drop · Rule validation

1 Introduction

Design and construction data usually consist of distributed heterogeneous information in proprietary serialized files leading to a separation of data. Building Information Modeling (BIM) was established as a method for centralized object-oriented information modeling to connect the existing data islands. The current approaches in the Architecture, Engineering, and Construction (AEC) industry focus on this centralized data repository in a building model [1], while Werbrouck et al. [2] recently showed, that the industry needs a more data-centric approach to connect distributed building data. Focusing on the data, these building models and the additional file-based planning data can be stored and linked within the Information Container for Data Drop (ICDD). This information container is based on recent Semantic Web technologies and works as a common environment for most file-based or model-based information, for example building models, images, spreadsheets or schedules. Within the container environment, the use of generic link sets adds a semantic layer so that information from the different sources can be linked together on an entity level. The semantically interlinked

heterogeneous resources generate the so-called linked building model, also referred to as nD-Model in industry.

However, planners often create and evaluate linked building models manually, because an automated traceable generation of links is not yet reasonably provided due to domain-specific file formats and cross-domain data in the unique construction processes [1]. Since construction projects require a large amount of information in several information models, the link sets between these models can also contain a large number of links, so that in practice it is not possible to maintain them manually. Semi-automatic validation is preferred in this paper to ensure the syntactic, semantic and technical correctness of link sets for the practical application of linked building models. The validation relies on a set of parametric rules on the one hand and on the building model, an information model, and the link set connecting both inside the information container on the other hand.

This paper provides a short overview of rule checking and validation approaches in the AEC industry and an overview of different rule languages for automated validation of linked building models using Semantic Web technologies. In the related work, Semantic Web-specific languages like the Semantic Web Rule Language (SWRL), the SPARQL Inferencing Notation (SPIN), and the Shapes Constraint Language (SHACL) are within the scope. Based on the literature, the present study introduces a methodology for applying rule-based validation in particular for linked building models inside the ICDD. For the evaluation, a case study shows a prototypic implementation of the methodology using an exemplary linked building model from the mechanized tunnel construction domain inside an ICDD. This building model is linked to a tunnel construction schedule utilizing a binary directed link set as linked construction schedule model (4D).

2 Related Work

The research on the use of rules for validation and code compliance of building models significantly advanced with the development of BIM and the Industry Foundation Classes (IFC). Mathematical logic defines rules as one paradigm to model and validate knowledge according to a certain domain of logic that can be, for instance, the first-order logic [3]. Rule languages enable the expression, exchange, and interpretation of rules [4]. Regardless of the rule language used, rule checks require formalization of the technical rule, a corresponding interpreter, a prepared building model, a processor that executes rules on a model, and an output of the results [5]. According to Schwabe et al. [6], the rule-based model checking is still expandable and the need for a specific rule language for the checking of building models was identified in recent research. The following chapter gives an overview of the literature on the general use of rule languages in the AEC industry, on the use of rule languages according to Semantic Web approaches and on the use of standardized linked building models referencing the multi-model approach and the ISO-standardized ICDD.

2.1 Use of Rule Languages in the AEC Environment

In terms of computer science, research and practice defines the information model or the amount of information to be checked as the fact base or knowledge base. Depending on the assumption of the completeness of the knowledge base, for the application of the rules to the facts, we distinguish between the validation of the knowledge base in the Closed World on the one hand and reasoning in the Open World on the other hand [7]. Reasoning accordingly describes the information gained from the application of rules. It generates additional information added to the knowledge base. This method is called rule-based reasoning or inferencing [8]. Solihin and Eastman published an overview of seven different application types of rule checking in the AEC environment and a classification into four different classes according to the complexity of the information needed within the facts to process the inference [9]. The classification of Solihin and Eastman distinguishes between a low requirement for explicit information needs, the need for derived information through model preparation, the need for extended data structures for testing, and the need for concept-related and solution-oriented checking modifying the facts to achieve a certain task.

Regarding the implementation of rule checking, three general approaches are considered: the use of hard-coded rules, the use of parametric approaches and the language-driven approach [5]. All three approaches have been investigated in research, while the third approach is most often used for building models. The suitability of the language-driven approach regarding BIM relies on its ability to express complex rules derived from natural language in vendor-neutral exchangeable rules. Hard-coded rules cannot be exchanged properly and parametric approaches do not always offer the ability to formulate complex scenarios. In the following, applications of rule languages in the construction industry and the respective rule languages in their context are presented.

In computer science, information modeling and software engineering are often performed using standardized diagrams from the Unified Modeling Language (UML). The Object Constraint Language (OCL) was developed to define boundary conditions in this information modeling with UML [10]. Boundary conditions in OCL are represented by invariants, which must be valid when the model is instantiated. For validation according to the semantic concept of BIM, Kraft [11] investigated how the use of the standardized constraint language OCL can support the compliance of building models with technical rules. Kraft concludes that the use of OCL as a formalized validation constraint is contradictory to the extendable concept of the Industry Foundation Classes (IFC).

Lee et al. [12] proposed a more BIM-specific language referred to as the Building Environment Rule and Analysis (BERA). This language is developed to deal with building information models from IFC data mapped into a specific object data model. BERA rules and the object data model are transferred into Java code and checked in a Java-based executor. Dimyadi et al. [13] proposed another example for BIM-specific use of rule languages as the BIM Rule Language (BIMRL), which focuses on IFC data and addresses the gap between the required information and the availability of information in the building information model.

Schwabe et al. [6] present another approach of rule-based model checking using the open-source rule engine Drools. Conclusively, they stated that even in current research on rule checking, there is a lack of satisfactory solutions that are open to interpretation and are not based on hard-coded rules. A promising approach for the non-proprietary definition of rules is the use of Semantic Web technologies discussed in the next section.

2.2 Rule Modeling and Checking Using Semantic Web Technologies

Semantic Web describes a technology in which the conventional unstructured data on the Web are enriched with structured data to make the meaning of the information machine-readable. The structuring of the data can be described by ontologies, which in turn are expressed in standardized language constructs based on the Resource Description Framework (RDF) and the Web Ontology Language (OWL) [3].

The RDF is a framework for information representation in the web recommended by the World Wide Web Consortium (W3C) [14]. The RDF is a modeling format for ontologies that creates interoperable data for both human perception and machine processing. OWL is a descriptive language for publishing and exchanging ontologies within the syntax of RDF and mainly extends the expressivity of RDF, e.g. with vocabularies for describing classes. At the abstract level, an ontology in the Web Ontology Language contains a collection of axiomatic facts in the subject-predicate-object pattern [3].

Rattanasawad et al. [4] published a general review of rule languages for the Semantic Web referring to the SWRL, which is a subtype of the declarative logic programming language Rule Modeling Language (RuleML) combined with the description logic of OWL. SWRL can be used to express rules as well as logic and can be written in either a human-readable syntax or RuleML-XML, respectively RDF-XML syntax [15]. The SWRL bases on the predicate logic declaring a body consisting of conditions and a head of consequent implications assuming an open world. This method is on the one hand compatible with the declaration of facts in OWL, but on the other hand, it can be ambiguous if a relation is not restricted by conditions [3]. Furthermore, Pauwels et al. [7] conducted a detailed investigation of the performance of rule languages in the Semantic Web area for use with IFC data. The research focuses on the rule checking using rules in the SWRL and SPARQL Inference Notation (SPIN). As a W3C Member Submission, SPIN is a syntax to represent rules and constraints using the SPARQL Protocol And RDF Query Language (SPARQL) that is a language to query or manipulate RDF-based data [16]. SPIN is used for both the inference of knowledge and the validation of constraints for existing facts. In their research, Pauwels et al. [7] showed that all of the examined languages and implementations have certain characteristics and cannot be compared directly, but can lead to certain conclusions for certain application cases. Zhang et al. [17] presented another approach of rule checking using SPARQL for real-world scale building models. It is based on a conversion of the building model into OWL and the transformation of natural language formalized rules into SPARQL queries on the model. The authors prove that rule compliance checking for large building models is possible in a reasonable time. Werbrouck et al. [2] presented the Shapes Constraint Language (SHACL), which is a

successor of SPIN, in a validation approach on distributed linked building data. They proposed to use SHACL to validate a distributed linked building model in a decentralized common data environment.

The SHACL is the most recent W3C Recommendation for the validation of RDF graphs, where one of the graphs in this context is the so-called data graph that is validated against a shapes graph including a set of conditions [18]. According to the W3C Recommendation, a shapes graph consists of target shapes that determine the resources to be validated and of node shapes and property shapes that specify conditions for either the target node or its properties. As SHACL is the successor of SPIN, it also supports the selection of SPARQL-based targets. The output of the validation process is a specific Validation Report specified in the SHACL Vocabulary.

2.3 Standardized Linked Building Models

Linked building models in a container-based environment have been researched especially by Fuchs et al. [1] as the multi-model approach and by Hoeber et al. [19] as the COINS container. The aim of the multi-model approach has been the cross-domain applicability using original models in a standardized, serialized format with persistent and restorable links and metadata. A multi-model is associated with one or more elementary models, which can be any instances of self-contained original data models, e.g. IFC models or schedules.

Furthermore, the multi-model can be associated with any number of linked models, each of which is represented by a non-empty set of links. These links consist of at least two or more linked elements from different elementary models. Each linked element belongs only to a single elementary model and contains the information to identify the corresponding element from the elementary model. The XML-serialized and ZIP-archived expression of the schema and the inherent original data are referred to as the multi-model container.

However, the COINS container is a more industry-driven development targeting the standardization of a flexible information container for linking a certain amount of building data from different domain models using linked data approaches. Like the multi-model container, the COINS container links models using generic extendable link sets. The metadata of the container and the link sets are serialized in RDF and can be exchanged in its entirety as a zip-archived file. The COINS container is the basis for the international standardization of the Information Container for Data Drop in the ISO 21597 [20].

3 Proposed Method

For the effective implementation of the rule-based validation of linked building models, several issues need to be addressed. Initially, to work with linked building models, the container format must be implemented to process the container into facts for validation. The processing of the container requires a library to deserialize and parse the RDF files from the container to get the required metadata information about the domain models and link sets. The second issue is the conversion of the documents from the original

data format into an OWL graph-based format to enable the validation using SHACL. After the conversion, the data need to be merged into a single data graph that can be validated by a shapes graph accordingly. Referring to the related work, these two issues can be summarized as the data preparation step. The third issue is the definition of the rules itself based on SHACL and with the inclusion of SPARQL. The rules have to be derived from natural language rules regarding the logical connection of the linked domain models. Corresponding examples are presented in the evaluation through the use case. The last issue to be solved is the actual validation with a SHACL engine.

According to the issues defined before, Fig. 1 shows the general process of the validation of linked building models in this approach. We assume that a linked building model in an ICDD has been properly created in the first process step. The resulting file is imported into the validation framework that was developed in previous research and presented in [21]. This validation framework is based on the latest version of the open-source library dotnetRDF¹ that provides an RDF parser, a SPARQL query processor, and a SHACL validation engine.

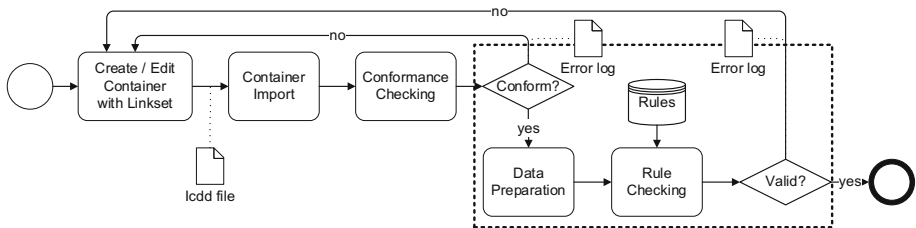


Fig. 1. Process of the validation of linked building models and scope (dotted) of this approach

After the import of the container file, the framework checks the conformance of the container according to the criteria of the underlying ISO standard. The whole process fails if the conformance cannot be checked and the container must be edited to fulfill the conformity until further validation is possible. The focus in this contribution lies on the consequent steps of the data preparation, the rule definition and, finally, the implementation of the rule checking process.

3.1 Data Preparation

Technically, domain models are defined and exchanged according to their underlying schemas. For instance, domain models can be geometry-based building models for design, structural engineering or MEP planning, but also alphanumeric models for cost planning or scheduling. To be able to check the individual models at the entity level, a conversion to a more generic format is necessary. In this approach, we choose RDF and OWL as a common data basis. The data preparation relies on the conversion from the

¹ <https://github.com/dotnetrdf/dotnetrdf>.

originally defined data schemes into RDF-based data, as depicted in Fig. 2. Therefore, the IFCtoRDF² converter is utilized for transferring building models from the IFC EXPRESS schema into the ifcOWL schema.

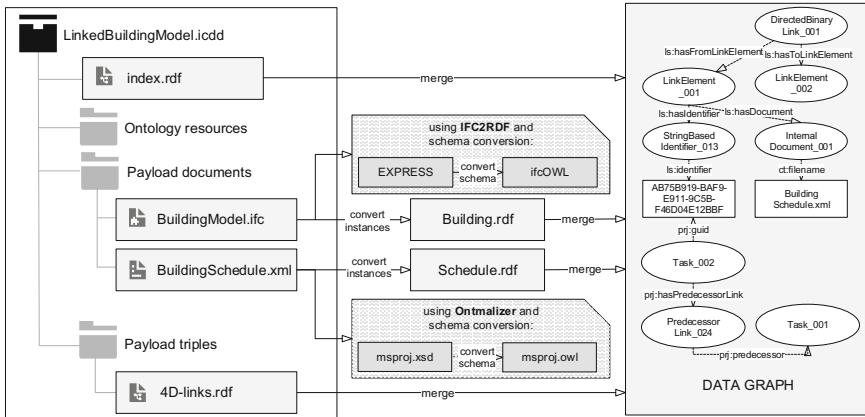


Fig. 2. Converting instances of domain models according to their schemas

Since the information container is not restricted to IFC-related file types, information from other text-based schemas need to be transferred to RDF. Examples are the MS Project-XML schema for project schedules or the GAEB DA XML (a specific format for the bill of quantities in Germany). Based on the respective schema file and the instance file that can be serialized with the schema, an OWL ontology and the corresponding instances can be converted to RDF instances. In this research, the open-source software Ontmalizer³ is used – a tool for transforming XSD and XML to OWL and RDF data. Ontmalizer relies on an XML schema object model library and the Apache Jena⁴ framework. The tool bases on a set of transformation rules that map the types defined in the XSD to respective types in OWL/RDF, for instance, the *xs:complexType* to *owl:Class* or an *xs:element* to either an *owl:DatatypeProperty* or an *owl:ObjectProperty* depending on the type of the element. Both, the IFCtoRDF converter and the Ontmalizer are utilized for the validation framework as compiled jar files. After the data is converted, the generated individual graphs of the models, the parent container graph and the link set graph are merged into a single data graph, which is exemplarily depicted in Fig. 2 indicating an extract from the RDF graph.

² <https://github.com/pipauwel/IFCtoRDF>.

³ <https://github.com/srdc/ontmalizer>.

⁴ <http://jena.apache.org/>.

3.2 Rule Definition

Before technical rules can be formalized between the instances of the models, the respective identifiers must first be determined and the structure of the link validated. Therefore, the RDF graph of a directed binary link inside a link set is considered in Fig. 3. The link consists of two linked elements, whereby one is the “from” element and the other one is the “to” element. Both elements have the mandatory relation to a document that is registered in the container and the optional relation to the identifier of the entity inside the respective models.

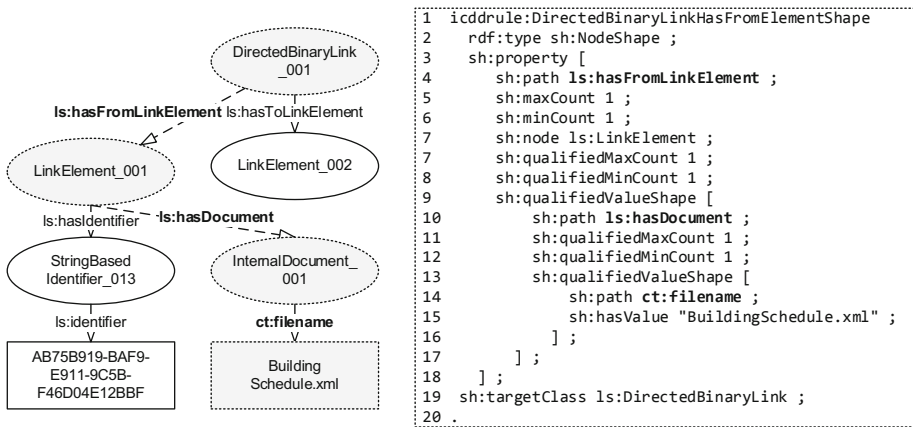


Fig. 3. RDF graph for a directed binary link in the ICDD and corresponding SHACL validation

In order to perform a validation based on the parameters of the entities inside the container, we need to prove whether the link elements have an identifier and whether the identifiers point to the entities of a model. If this is the case, the corresponding entities can be queried from the data graph so that access to the respective parameters is given.

The rule is defined in SHACL, therefore the vocabulary from the SHACL namespace is considered. The core vocabulary of the language has built-in constraint components on nodes like cardinality, value type, value range, and allowed values. Moreover, it is possible to define types of constraints for more complex validation conditions, for example with using SPARQL queries in SHACL shapes. A SHACL shape can be defined as an instance of a *sh:NodeShape*. A *sh:targetClass* can be applied to a *sh:NodeShape* to determine for which instances from the specified classes in the RDF graph the condition must be fulfilled.

On the right, Fig. 3 shows a rule for the validation which corresponds with the RDF graph on the left. The considered predicates in the rule and in the graph are highlighted with bold font and dotted nodes. This rule validates whether each node of the type *ls:DirectedBinaryLink* is related via an *ls:hasFromLinkElement* predicate to an *ls:LinkElement* node. This node must refer to a document with a defined file name or alternatively the URI of the document node. This concatenated relation over several

predicates can be modeled with the help of a *sh:qualifiedValueShape*. This type of shape restricts the path by using the object node of the previous validation as subject node for a subsequent validation. The occurrences of the subsequent path can also be restricted using *sh:qualifiedMinCount* and *sh:qualifiedMaxCount*. The identifiers of the link elements can be validated through the path-related validation using the *ls:hasIdentifier* predicate in the same way.

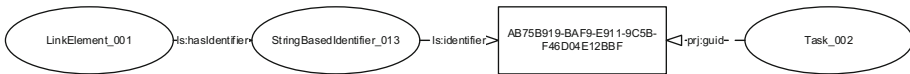


Fig. 4. The path from the identifier of a link to the identifier in the original model

In order to access the original model data for each linked entity for the validation, the respective identifiers from the link element must first be matched with the identifiers in the model data. As shown in Fig. 4, both predicates *ls:identifier* and *prj:guid* point to the same literal node containing the identifier. The modifier *sh:inversePath* is required to adjust the path-based validation accordingly. This modifier can be used in the *sh:path* property of a shape combined with a regular predicate. As a consequence for the validation, the focused node is not anymore treated as a subject, but as an object that must have exactly one incoming triple consisting of inverted regular path and a subject node. According to Fig. 5, this subject node is the linked entity in the original model. The parameters of this entity can be incorporated for semantic validation.

Comprehensive possibilities for validation can be developed on this basis. Considering a linked construction schedule model (4D), the predecessor of a task can be determined by developing a *sh:qualifiedValueShape* with the vocabulary from the project schedule namespace (*prj*) defined during the conversion of the schedule XML schema. The predicates *proj:hasPredecessorLink* and *proj:predecessor* can be used with the qualified cardinalities to find a preceding task. This shape can be used, for example, to check for a particular validation case whether the time between the end time of the predecessor and the start time of the focused task is less than a determined value.

Besides the core constraint components, the W3C SHACL recommendation offers the possibility of defining SPARQL constraint components to express even richer constraints. These constraint components can be defined once and be reused for several SHACL shapes to ensure the usability of the rules. The components define a constraint for a focus node that is validated based on a *SELECT* query delivering a result set with entries that do not fulfill the constraint.

3.3 Rule Checking

The precondition for the rule check is the existence of both a well-formed data graph and a valid shapes graph. Both are generated in the previous steps of data preparation and rule definition, whereby the well-formedness at the time of implementation is checked only in the validation engine. The validation engine used in this approach is

implemented in the dotnetRDF framework (see chapter 3). The framework supports the SHACL core implementation as well as the SHACL-SPARQL implementation and is successfully tested within the W3C SHACL Test Suite⁵ as a fully compliant engine.

The validation engine is implemented in a class *IcddRuleValidator* that is instantiated from an existing *IcddContainer* object and the path to the Turtle serialized shapes file. During the creation of the validator, the data graph is generated from the container as described in Sect. 3.1. The validation results are transformed from the single validation report provided by the dotnetRDF framework into an *IcddValidationResult* for each violation of the shapes to be displayed separately for each focus node.

4 Case Study and Evaluation

The validation of linked building models according to the proposed method is performed through a case study presented in the following. Figure 5 presents an exemplary tunnel model in the IFC 4 version and a corresponding construction schedule in an MS Project XML. The tunnel model, the construction schedule and an Information Container for Data Drop comprising these two models were created for demonstration purposes. A link set has been added to the container, which contains a directed binary link for each task, linking the Globally Unique Identifiers (GUID) of the tunnel rings and the tasks. This setup serves as a minimal example and considers only the order in which the tunnel rings are built.

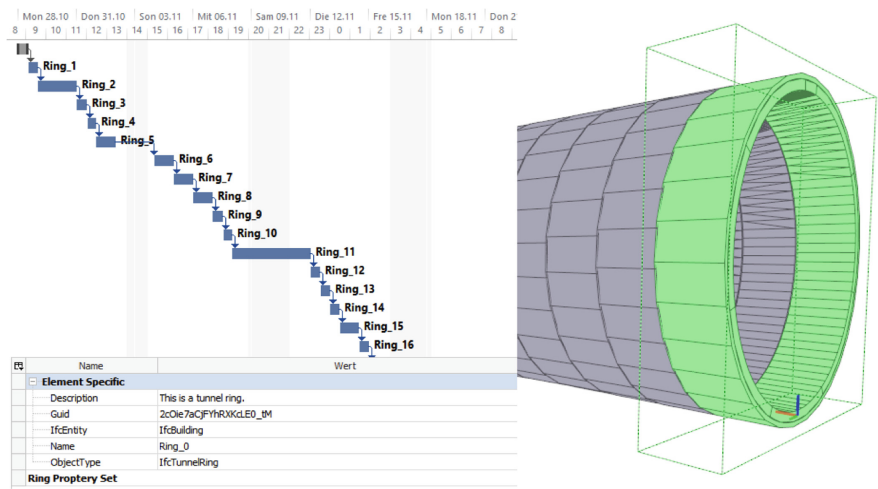


Fig. 5. Use case data: tunnel schedule (left) and tunnel model (right)

⁵ <https://w3c.github.io/data-shapes/data-shapes-test-suite>.

It is assumed, that each element in the schedule is assigned at least to one element in the building model, but not every element in the building model must have an assigned element in the schedule. This may be the case, for example, for proxy elements grouping a set of elements. Therefore, the usage of the *DirectedBinaryLink* type for the relation between schedule and building model is established. In this case study, the following boundary conditions should be checked in the linked building model:

1. **Conformity level:** For each link element that points to a specific GUID, an associated entity from the two original models must exist in the RDF graph.
2. **Semantic level:** Tunnel rings that are built directly in succession according to the schedule must be adjacent in the building model.

Since the conformity condition has already been defined as a general rule for each linked building model in Sect. 3.2, the focus of the rule design in the next section is on semantic level condition.

4.1 Rule Design

For the rule design, we use an extract from the aggregated RDF data graph shown in Fig. 6 and consider the *DirectedBinaryLink_001* as the focus node. The rule is developed as a SPARQL-based constraint and targets all directed binary links in the link set. In the *SELECT* query of the constraint, the focus node can be used with the pre-bound variable *\$this* when integrating the query into the shape.

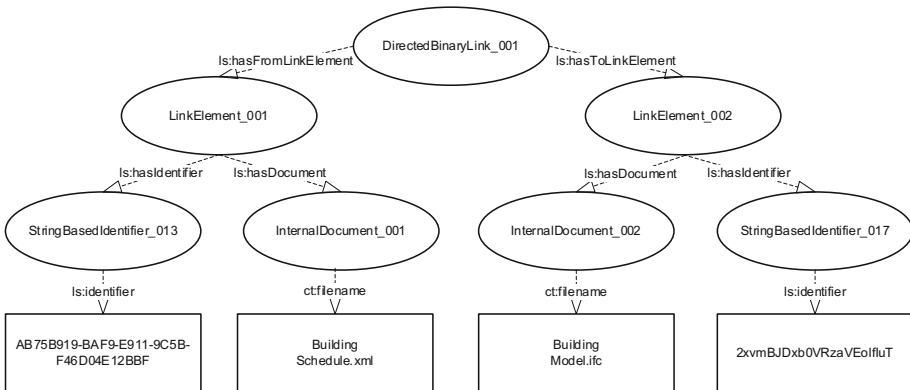


Fig. 6. Extract of a link instance from the data graph

First, the rule defined in natural language must be transferred and formalized. Therefore, all entities required for the examination of the rule must be consulted, which are the two preceding tasks from the schedule and the two adjacently expected building elements from the building model.

Beginning with the access of the tasks, we provide a query via the path from the *LinkElement_001* querying the respective string-based identifier as a literal node. The

associated task can be found inverting the query and considering the path *prj:guid* pointing to the same identifier as the *StringBasedIdentifier_013* (see Fig. 7). We get the individual of the task and can retrieve the corresponding predecessor for it, by querying the predecessor link as depicted in Fig. 7. Using the predecessor task (*Task_001*) and its GUID, the corresponding link (*DirectedBinaryLink_002*) in the link set can be determined. We have, then, determined both task individuals as well as the link elements and links that refer to these tasks.

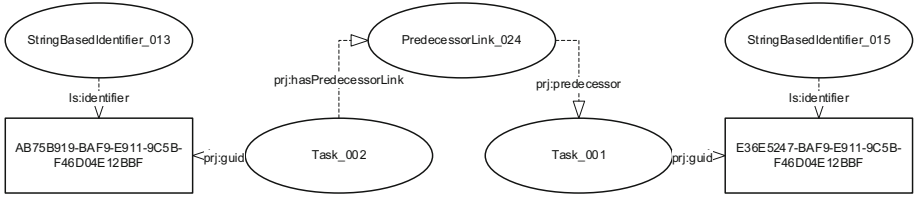


Fig. 7. Extract of the relation between two tasks in the data graph

The directed binary links are instrumentalized for the identification of the associated building elements in the building model. The linked building entity can be determined from the link by querying the path of the *hasToLinkElement* predicate. Following the path of the identifier, the respective global identifier from the IFC schema lead to the two individuals of the building elements *IfcBuildingElementProxy_2447* and *IfcBuildingElementProxy_2475*. Using the hierarchy from the IFC, the Cartesian coordinates of the two considered tunnel rings can be evaluated using the predicates *ifcowl:objectPlacement_IfcProduct*, *ifcowl:relativePlacement_IfcLocal*

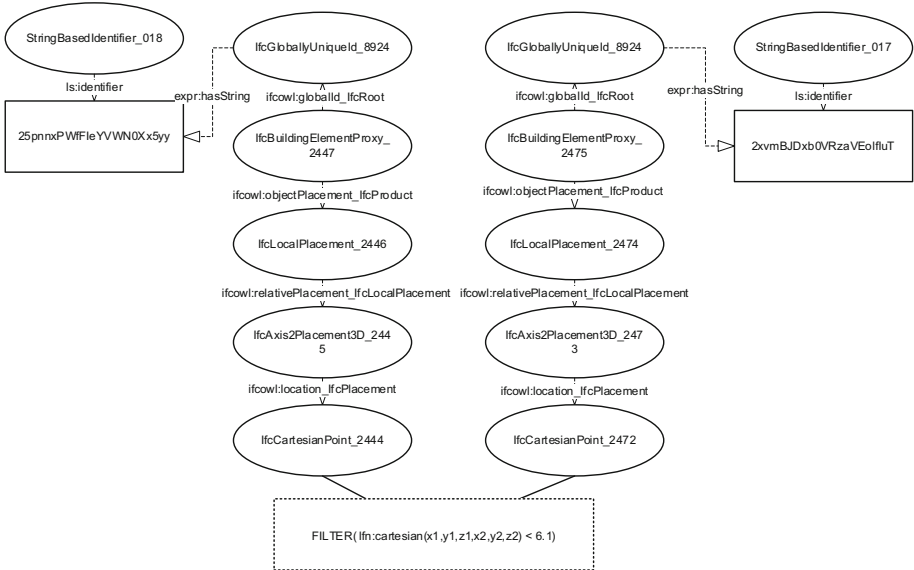


Fig. 8. Extract of the relation between links and building elements

Placement, *ifcowl:location_IfcPlacement* and *ifcowl:coordinates_IfcCartesianPoint* (see Fig. 8). In a last step, the x, y and z coordinates of each element need to be queried.

With the two Cartesian points and the access to the respective coordinates, the distance between these can be determined in a SPARQL expression. Unfortunately, neither the mathematical square root function nor the Cartesian distance is defined in the vocabulary of SPARQL. For this reason, an external vocabulary providing this function must be included. To extend the basic query vocabulary, dotnetRDF has implemented a so-called Leviathan Function Library⁶, which contains a variety of numerical extension functions for SPARQL and can be used in queries executed with dotnetRDF. Furthermore, the framework allows developers to implement custom SPARQL expressions for the integration into the query engine. The Cartesian distance between the two tunnel rings in a 3D coordinate system is determined by means of the numerical function *lfn:cartesian(x1, y1, z1, x2, y2, z2)* utilized from this library. In the last step, the distance needs to be compared to the fixed maximum value of the average width of a tunnel ring in the model (here 6.1 m).

Task	LinkedBuildingElement1	hasPredecessor	LinkedBuildingElement2	distanceBetween
prj:Task_002	inst:lfcBuilding_2437	prj:Task_001	inst:lfcBuilding_2369	5.999999999999999e0

Fig. 9. Evaluation of the SPARQL query on the data graph for a specified link

The query defined in SPARQL is tested for correctness before defining the rule on the aggregated dataset and returns a result set including the distance of two consecutive elements as shown in Fig. 9. For the integration into the SHACL constraint, the query is modified using a *FILTER* expression to return only results if the distance between tunnel rings is larger than 6.1 m. This means that the result set will be empty for all positive results and will pass the positive result to the SHACL constraint. The query is nested in the shape defined with an anonymous instance of *sh:sparql* constraint and put into the constraint with the *sh:select* property (see List. 1).

```

1 @prefix rule: <http://hive.vm.rub.de/semweb/icdd/rules#> .
2 @prefix sh: <http://www.w3.org/ns/shacl#> .
3 rule:DirectedBinaryLinkDistanceBetweenPredecessors
4   a sh:NodeShape;
5   sh:targetClass ls:DirectedBinaryLink;
6   sh:sparql [
7     a sh:SPARQLConstraint;
8     sh:message "The examined BuildingElements cannot be adjacent";
9     sh:select "" #SELECT QUERY INSERTED HERE "";
10  ] .

```

Listing 1: SHACL shape

⁶ <https://github.com/dotnetrdf/dotnetrdf/wiki/DeveloperGuide-SPARQL-Leviathan-Functions>.

4.2 Rule Checking and Results

The previously defined rule shape is transferred into the rule engine together with the aggregated data graph. For the validation to take effect, the first link in the link set was connected to a tunnel ring that is not adjacent to the tunnel ring of the predecessor. The result in Listing 2 shows, that the respective link failed the validation as it caused a violation of the defined shape. The validation report with the predicate *sh:conforms* (line 2) states that during the validation violations of the shapes have occurred. These violations are displayed in *sh:result* specifically for each checked node and described according to the defined rule (line 3–10).

```
1 _:autos1 a sh:ValidationReport;
2 sh:conforms false;
3 sh:result [a sh:ValidationResult;
4   sh:focusNode links:DirectedBinaryLink_001;
5   sh:value links:DirectedBinaryLink_001;
6   sh:resultMessage "The examined BuildingElements cannot be adjacent";
7   sh:sourceConstraintComponent sh:SPARQLConstraintComponent;
8   sh:sourceShape   rules:DirectedBinaryLinkDistanceBetweenPredecessors;
9   sh:resultSeverity sh:Violation;
10  sh:sourceConstraint []].
```

Listing 2: SHACL validation result set

5 Discussion

The case study presented in this paper has confirmed, that available technologies make it feasible to query and validate linked building models. Although the objective of this research was achieved, the simplified case study has some limitations, as it covers only an exemplary technical rule for the validation of linked building models.

Due to the highly complex relationship between the connected models, the reusability of rules cannot be assumed, and a user-supported adaptation for each application is required. Therefore, a fully automated checking of linked building models for the correctness of the links is not possible at this state of research so that there is only a human-assisted semi-automated checking realized.

This research has disclosed that the use of Semantic Web technologies, especially RDF, SPARQL, and SHACL, provide an advantage over hard-coded rule checking, as rules can be exchanged easily and can be included directly within the information container. The combination of SPARQL and SHACL offer a wide range of possibilities for specifying rules for validation. However, this can be problematic when formulating rules manually, as there is a high potential for errors. Furthermore, the exact knowledge of the RDF graphs of the respective data models is required to formulate appropriate rules.

6 Conclusion and Outlook

Although linked building models are widely used in practical applications, especially for 4D or 5D use cases, the creation and the exchange of those models, e.g. in an information container, is hardly considered either by theory or praxis. This research focused on an easier creation and exchange of linked building models providing a method to validate the correctness of the defined links. As the related work shows, there is a significant number of research approaches regarding rule checking for the construction industry and especially approaches based on building models. This focus should be extended to include rule checking for linked models and thus, for all information within the planning process.

This paper demonstrates the feasibility of rule-based validations in the information container by processing all contained models into RDF and validate the data in a rule engine according to SHACL shapes. This paper demonstrates how natural language rules can be formalized as SHACL shapes for link sets based on the parameters of linked entities of the original models.

Generally, a benchmark with real-world models and different use cases should be carried out to make a dedicated statement about the usability of the method. With regard to the conversion of data into an RDF graph, a long process time can occur with large models and models with many documents. Therefore, it is suitable to implement simplifications for further development, e.g. to consider existing approaches to streamline IFC models in ifcOWL or to remove unnecessary information like the geometry representation from the models.

However, the proposed method shows that the Semantic Web is a valuable basis for performing comprehensive validation. Nevertheless, this approach is not yet practicable for use in construction practice, since the rule definition requires knowledge of Semantic Web technologies as well as the respectively used data schemas to convert natural language rules into formalized rules related to the information container. In order to bridge these knowledge gaps, an extension of the vocabulary with ICDD-specific predefined constructs and simplifications, such as the BIMSPARQL extension, will be considered. The aim for further research is the increase of the usability of the ICDD for practical applications. A further important point is the definition of a simple language for the representation of rules in natural language that can be translated into a conform SHACL-SPARQL constraint component.

References

1. Fuchs, S., Scherer, R.J.: Multimodels — instant nD-modeling using original data. *Autom. Constr.* **75**, 22–32 (2017). <https://doi.org/10.1016/j.autcon.2016.11.013>
2. Werbrouck, J., Pauwels, P., Beetz, J., van Berlo, L.: Towards a decentralised common data environment using linked building data and the solid ecosystem. In: *Proceedings of the 36th CIB W78 2019 Conference*, Northumbria University, Newcastle (2019)
3. Hitzler, P., Krötzsch, M., Rudolph, S.: *Foundations of Semantic Web Technologies*. CRC Press/Chapman/Hall/CRC textbooks in computing, Boca Raton/London/New York (2010)

4. Rattanasawad, T., Saikaew, K.R., Buranarach, M., Supnithi, T.: A review and comparison of rule languages and rule-based inference engines for the semantic web. In: Proceedings of the 2013 International Computer Science and Engineering Conference (ICSEC), pp. 1–6 (2013). <https://doi.org/10.1109/icsec.2013.6694743>
5. Eastman, C., Lee, J.-m., Jeong, Y.-s., Lee, J.-K.: Automatic rule-based checking of building designs. *Autom. Constr.* **18**(8), 1011–1033 (2009). <https://doi.org/10.1016/j.autcon.2009.07.002>
6. Schwabe, K., Teizer, J., König, M.: Applying rule-based model-checking to construction site layout planning tasks. *Autom. Constr.* **97**, 205–219 (2019). <https://doi.org/10.1016/j.autcon.2018.10.012>
7. Pauwels, P., de Farias, T.M., Zhang, C., Roxin, A., Beetz, J., de Roo, J., Nicolle, C.: A performance benchmark over semantic rule checking approaches in construction industry. In: *Advanced Engineering Informatics*, vol. 33, pp. 68–88 (2017). <https://doi.org/10.1016/j.aei.2017.05.001>
8. De Meester, B., Heyvaert, P., Arndt, D., Dimou, A., Verborgh, R.: RDF graph validation using rule-based reasoning. *Semant. Web* **1**, 1–5 (2020)
9. Solihin, W., Eastman, C.: Classification of rules for automated BIM rule checking development. *Autom. Constr.* **53**, 69–82 (2015). <https://doi.org/10.1016/j.autcon.2015.03.003>
10. Cabot, J., Gogolla, M.: Object Constraint Language (OCL): a definitive guide. LNCS, vol. 7320 (2012). https://doi.org/10.1007/978-3-642-30982-3_3
11. Kraft, B.: Using object constraint language for the validation of BIM platform object graphs. In: *Proceedings of the Lake Constance 5D-Conference 2013*, vol. 220, pp. 90–98 (2013)
12. Lee, J.-K., Eastman, C.M., Lee, Y.C.: Implementation of a BIM domain-specific language for the building environment rule and analysis. *J. Intell. Robot. Syst.* **79**(3–4), 507–522 (2015). <https://doi.org/10.1007/s10846-014-0117-7>
13. Dimyadi, J., Solihin, W., Eastman, C., Amor, R.: Integrating the BIM rule language into compliant design audit processes. In: *Proceedings of the 33rd CIB W78 Conference 2016, Brisbane* (2016)
14. Cyganiak, R., Wood, D., Lanthaler, M.: RDF 1.1 Concepts and abstract syntax, W3C Recommendation 25 February 2014
15. Horrocks, I., Patel-Schneider, P.F., Boley, H., Tabet, S., Grosz, B., Dean, M.: SWRL: a semantic web rule language combining OWL and RuleML. W3C Memb. Submiss. **21**(79), 1–31 (2004)
16. Knublauch, H., Hendler, J.A., Idehen, K.: SPIN - overview and motivation. W3C Member Submission 22 February 2011 (2011)
17. Zhang, H., Zhao, W., Gu, J., Liu, H., Gu, M.: Semantic web based rule checking of real-world scale BIM models: a pragmatic method. In: *Proceedings of the International Congress and Conferences on Computational Design and Engineering (i3CDE'19), Penang, Malaysia*, pp. 130–137 (2019)
18. Knublauch, H., Kontokostas, D.: Shapes Constraint Language (SHACL). W3C Recommendation 20 July 2017 (2017)
19. Hoeber, H., Alsem, D.: Life-cycle information management using open-standard BIM. *Eng. Constr. Archit. Manag.* **23**(6), 696–708 (2016). <https://doi.org/10.1108/ECAM-01-2016-0023>
20. ISO 21597-1, 2018: Information Container for Data Drop - Exchange specification - Part 1: Container (ISO/DIS 21597-1:2018), German and English version prEN ISO 21597-1 (2018)
21. Hagedorn, P.: Implementation of a validation framework for the information container for data drop. In: *Proceedings 30, Forum Bauinformatik, At Bauhaus Universität Weimar* (2018)